Refactoring Evidence

BEFORE

```java
// GRASP: Creator - Service creates Missile objects
public MissileDTO createMissile(String name, String type, double price, int quantity) {  6 usages  willj067
    if (name == null || name.isBlank()) {
        throw new IllegalArgumentException("Missile name cannot be empty");
    }
    if (price <= 0 || quantity <= 0) {
        throw new IllegalArgumentException("Price and quantity must be greater than zero");
    }

    Missile missile = new Missile(name, type, price, quantity);
    Missile saved = missileRepository.save(missile);
    return toDTO(saved);
}
```

AFTER

```java
public class MissileService {

    private final MissileRepository missileRepository;
    private final Map<String, LaunchMethod> launchMethodMap;

    // SOLID: SRP - validator has its own responsibility
    // DRY: Centralized validation reused here
    private final MissileValidator missileValidator = new MissileValidator();  1 usage

    // GRASP: Creator - Service creates Missile objects
    // SOLID: SRP - createMissile only handles creation, not validation
    public MissileDTO createMissile(String name, String type, double price, int quantity) {  6 usages
        // DRY: Reusing centralized validation
        missileValidator.validate(name, price, quantity);

        Missile missile = new Missile(name, type, price, quantity);
        Missile saved = missileRepository.save(missile);
        return toDTO(saved);
    }
```

Explanation

BEFORE: Validation logic was embedded inside MissileService.createMissile(), violating SRP — the Service was responsible for both business logic and validation.

AFTER: Validation was moved to a separate MissileValidator class. MissileService now only handles business logic, following the Single Responsibility Principle (SRP).